DATA FLOW ARCHITECTURE

The telecom data pipeline follows a structured, layered architecture designed for scalability, reliability, and maintainability.

1. Data Ingestion Layer

Raw telecom CSV files are placed in the data/landing/ directory.
Apache Airflow orchestrates ingestion using a DAG pipeline.
The DAG performs:
-File detection
-Schema validation
-File classification (valid / invalid)
-Movement to appropriate zones

2. Raw Data Layer

Valid files are moved to data/raw/.
Invalid files are moved to data/rejected/.
This layer:
-Preserves original data without modification
-Ensures traceability and auditability
-Supports reprocessing if needed

3. Processing Layer (PySpark ETL)

Spark processes data using telecom_pipeline.py.
Operations include:
-Schema standardization
-Data cleaning and filtering
-Aggregations and transformations
-Feature extraction
Processing is executed as part of the Airflow DAG using:
run_spark_job()

4. Processed Data Layer

Cleaned data is stored in: data/processed/
Format: Parquet
Partitioned by: day (date-based partitioning)
Benefits:
-Optimized storage
-Faster analytical queries
-Distributed processing compatibility

5. Data Warehouse Layer

Processed data is loaded into a SQL warehouse.
Uses Star Schema for analytics:
-Fact table: fact_usage
-Dimensions: dim_time, dim_region
Loading is handled by:
load_warehouse()

6. Orchestration Layer (Airflow)

Airflow DAG manages the full pipeline:
detect_files
→ validate_files
→ move_files
→ log_status
→ run_spark_job
→ load_warehouse
→ notify
Ensures:
-Task dependency management
-Pipeline automation
-Fault tolerance

7. API Layer

FastAPI exposes warehouse data via REST APIs:
/usage/summary
/usage/region/{region}
/usage/peak
/usage/features/{region}
/predict-usage-risk
Enables real-time data access

8. Machine Learning Layer

Feature engineering based on processed data
Model (Random Forest) predicts:
-Congestion risk
-Anomaly detection
Predictions served via API

9. Frontend Layer

React dashboard consumes API
Provides:
-Usage analytics
-Regional insights
-Peak traffic visualization
-Risk prediction interface

Summary:
This layered architecture ensures:
-Scalability (Spark + Parquet)
-Automation (Airflow)
-Realtime access (API + UI)
-Predictive intelligence (ML)

Schema Design

The system uses a Star Schema for efficient analytical querying.

Fact Table: fact_usage
ColumnDescriptionusage_idPrimary keytime_idForeign key to dim_timeregion_idForeign key to dim_regioncall_countTotal callssms_countTotal SMSinternet_mbInternet usage
Dimension Table: dim_time
ColumnDescriptiontime_idPrimary keydateFull datehourHour of daydayDaymonthMonthweekdayDay name
Dimension Table: dim_region
ColumnDescriptionregion_idPrimary keyregion_nameRegioncityCity

Design Rationale
-Star schema simplifies complex queries
-Enables fast aggregations
-Reduces redundancy vs flat tables
-Optimized for BI and analytics

Task 3.3 — Batch vs Streaming Decision

Data Flow	Type	Explanation
Network activity logs from cell towers	Streaming	Real-time telecom data generation
Daily usage summary	Batch	Daily aggregated usage processing
Monthly billing reports	Batch	Monthly billing report generation
Network congestion alerts	Streaming	Instant network congestion alerts
Customer dashboard data	Near Real-time / Hybrid	Near real-time dashboard updates

Explanation

Telecom systems generate a mix of real-time and historical data.
- Streaming data is used where low latency is critical (e.g., network monitoring, alerts).
- Batch processing is suitable for periodic reporting such as billing and summaries.
- Some use cases like dashboards use a hybrid model (micro-batching or near real-time).
This design ensures a balance between performance, cost, and system complexity.

Task 3.5 — Storage Strategy

Raw Layer (data/raw/) — Why CSV?
The raw layer stores incoming telecom data in CSV format.
Reasons:
- CSV is a simple, widely supported format and easy to ingest from multiple sources.
- It preserves the original structure of incoming data without modification.
- Suitable for initial storage where data is not yet processed.
- Allows easy debugging and auditing since it is human-readable.
Thus, CSV is ideal for the raw ingestion layer where data is stored as-is.

Processed Layer (data/processed/) — Why Parquet and Partitioning?
The processed layer stores cleaned and transformed data in Parquet format.
Reasons for using Parquet:
- Columnar storage reduces disk usage and improves query performance.
- Supports efficient compression and encoding.
- Optimized for big data frameworks like Spark.
- Enables faster analytical queries compared to row-based formats.
- Supports predicate pushdown for efficient filtering
Reasons for partitioning by date:
- Allows efficient filtering for time-based queries.
- Reduces the amount of data scanned during queries.
- Improves performance for aggregation tasks.
- Supports scalable data storage as data volume grows.
Thus, Parquet with partitioning is ideal for optimized analytical workloads.

Warehouse Layer — Why Star Schema?
The warehouse uses a star schema consisting of one fact table (fact_usage)
and dimension tables (dim_time and dim_region).
Reasons:
- Simplifies complex queries by separating facts and dimensions.
- Improves query performance through smaller joins.
- Supports efficient aggregations for business metrics.
- Provides a clear and scalable data model.
- Reduces data redundancy compared to flat tables.

Compared to a single flat table:
- A flat table leads to duplication of data.
- Star schema provides better maintainability and flexibility.
- Enables efficient OLAP-style analytics.
Thus, a star schema is chosen for its performance and analytical capabilities.

Task 3.6 — Failure Handling

Airflow provides retry and task-level failure isolation for robust execution.

1. Missing Input File

Scenario:
No CSV files are present in the data/landing directory.
Observed Behavior:
- The detect_files() task returns an empty list.
- The validate_files() task processes no files.
- The Spark job is either skipped or fails gracefully with a controlled message.
Handling:
The pipeline checks for file presence before processing.
If no files are found, the pipeline logs a warning and prevents unnecessary execution.
Conclusion:
The pipeline handles missing input data safely without crashing.

2. Corrupt or Invalid CSV File

Scenario:
A CSV file with incorrect schema (missing or extra columns) is placed in data/landing.
Observed Behavior:
- The validate_files() task compares file headers with expected schema.
- Invalid files are identified correctly.
- These files are moved to data/rejected directory.
Handling:
Schema validation ensures that only clean and compatible data is processed.
Invalid files are isolated for further inspection.
Conclusion:
The pipeline prevents bad data from affecting processing by isolating corrupt files.

3. Duplicate File Handling

Scenario:
The same file is uploaded multiple times into data/landing.
Observed Behavior:

- Files are detected by name.
- Duplicate files can be processed again if not handled explicitly.
Handling:
Duplicate detection can be implemented using filename checks or hashing.
In the current pipeline, duplicates follow the same ingestion flow.
Improvement Suggestion:
Enhance the pipeline by:
- Maintaining a processed file log
- Skipping already processed files
Conclusion:
The pipeline can be extended to avoid duplicate processing using tracking mechanisms.

Summary

The pipeline is designed to handle real-world data issues such as missing data,
corrupt files, and duplicate ingestion scenarios.
These mechanisms improve reliability, data quality, and robustness of the system.

Task 6.1 — Problem Statement

The goal is to predict telecom network congestion risk based on usage patterns.
Congestion risk occurs when network demand exceeds capacity, leading to degraded service quality.
Predicting congestion helps telecom operators proactively manage network load, optimize resources, and improve user experience.
Since the dataset does not contain labels, a rule-based labeling strategy is used:
- If usage is above the 90th percentile → HIGH risk
- If usage is between 70th–90th percentile → MEDIUM risk
- Otherwise → LOW risk
This allows transforming unlabeled data into a supervised learning problem.
The model uses features such as average usage, growth rate, variability, and peak usage patterns to predict congestion and detect anomalies. This enables predictive analytics capability within the pipeline, transforming it from a descriptive system into a decision-support system.